

Architecture Conformance Checking in Microservice-based Systems: A Systematic Review

Autores:

Ednilson Geraldo Rossi^{1, 2}, Valter Vieira de Camargo², e Daniel San Martín³

¹Federal Institute of São Paulo, Araraquara, Brazil

²Federal University of São Carlos, São Carlos, Brazil

³Universidad Católica del Norte, Coquimbo, Chile

Author's Accepted Manuscript (Post-print)

This is the author's accepted version of the paper to be published in the **Proceedings of the Brazilian Symposium on Software Quality (SBQS 2026)**.

© 2026 SBC – Sociedade Brasileira de Computação.

Personal use of this material is permitted. Permission from SBC must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

The official version will be available via the SBC OpenLib (SOL) platform.

Architecture Conformance Checking in Microservice-based Systems: A Systematic Review

Ednilson Geraldo Rossi
Federal Institute of São Paulo
Federal University of São Carlos
São Carlos, São Paulo, Brazil
ednilsonrossi@ifsp.edu.br

Valter Vieira de Camargo
Federal University of São Carlos
São Carlos, São Paulo, Brazil
valtervcamargo@ufscar.br

Daniel San Martín
Universidad Católica del Norte
Coquimbo, Chile
daniel.sanmartin@ucn.cl

Abstract

Architectural Conformance Checking (ACC) is an architectural quality assurance technique used to verify whether an *implemented architecture* (IAr) conforms to architectural rules defined in a *planned architecture* (PAr), enabling the identification of architectural violations. Although ACC has been extensively studied in the context of monolithic systems, it remains unclear how it is conducted for microservice-based systems (MBSs). Aspects such as distributed and asynchronous communication, independent deployment, data ownership boundaries, and technology heterogeneity makes ACC for MBS much more challenging. Therefore, we have conducted a Systematic Review (SR) to characterize the state of the art in this context following well-established guidelines from literature. We have run the SR in January 2026 and reached a final set of 28 primary studies. The results indicate that: (i) architectural rules are dealt and specified in a higher level than traditional ACC, mainly related to patterns and design principles; (ii) the recovering techniques of IArS employ a combination of heterogeneous and distributed artifacts; and (iii) ACC in MBSs extends traditional structural verification towards a more comprehensive assessment of architectural quality.

Keywords

Architectural Rules, Software Architecture, Architectural Quality

1 Introduction

Microservice-based systems (MBS) have become one of the dominant architectural styles for modern distributed applications. Their fundamental principle consists of decomposing an application into independent, loosely coupled, and specialized services that communicate through distributed mechanisms [27, 30, 48]. Some of their main characteristics are distributed and asynchronous communication, independent deployment, event-driven dependencies, data ownership boundaries and technology heterogeneity. It promotes quality properties such as scalability, flexibility and parallel development by autonomous teams, leading to its widespread adoption in both industry and academia [48].

Architectural Conformance Checking (ACC) is a well known quality assurance technique for identifying architectural drifts of a system by comparing its implementation, usually referred as Implemented Architecture (IAr), with the *architectural rules* defined on its Planned Architecture (PAr) [4, 21]. Its primary objective is to identify architectural violations and inconsistencies introduced during software maintenance [14, 15, 23]. Although ACC has been extensively studied in the context of monolithic systems [11, 23, 24, 29, 38, 50], it remains unclear how ACC has evolved to address the characteristics of MBSs.

In monolithic systems, ACC predominantly focuses on verifying structural rules among logical architectural elements (layers, modules, components, filters) that live within the same application. In this context, the IAr can be recovered primarily from the source code, enabling its comparison with the PAr [21, 23, 29]. However, MBSs introduce characteristics that make this process significantly more complex [27, 30].

Although there exist some approaches for ACC in MBSs, the literature remains fragmented. There is still no comprehensive understanding of which architectural rules are verified, how these rules are represented, how the IAr is recovered and represented, or how conformance checking is performed. This lack of a systematic characterization hinders a comprehensive understanding of how ACC has evolved with respect to the rules being considered, the recovery and representation of the implemented architecture, and the conformance checking mechanisms themselves [42].

To address this gap, this paper presents a Systematic Literature Review (SLR) aimed at characterizing the state of the art of ACC for MBSs. We have conducted an analysis considering three complementary perspectives of ACC: (i) the architectural rules and their forms of specification; (ii) the methods and artifacts employed to recover the IAr; and (iii) the mechanisms used to perform conformance checking. To guide the conduction of the SLR, three research questions were defined, following established methodological guidelines [22, 52]. During the primary study selection phase, inclusion and exclusion criteria were applied with the support of Large Language Models (LLMs) as an aid for screening [18]. This process initially identified 19 studies from digital libraries, which were complemented by 9 additional studies obtained through backward and forward snowballing, totaling 28 analyzed primary studies.

The results reveal that ACC in MBSs differs from traditional ACC. First, architectural rules evolve from structural dependency constraints toward higher-level architectural properties, such as service autonomy, communication, resilience, and loose coupling. Second, recovering the IAr requires combining multiple heterogeneous artifacts beyond source code, including deployment descriptors, configuration files, API specifications, and runtime information. Finally, ACC itself evolves from structural consistency verification toward the assessment of architectural quality in MBSs.

Beyond providing a comprehensive characterization of the state of the art of ACC for MBSs, this work proposes a classification scheme for architectural rules based on four complementary dimensions: *i) Categories* (What?), which capture the architectural concept and purpose addressed by a rule; *ii) Representations* (How?), which describe how rules are specified or represented; *iii) Architectural Knowledge Sources* (Where from?), which identify the architectural

artifacts or knowledge sources from which rules are derived; and *iv*) *ACC Scopes* (Where?), which indicate the ACC scope in which rules are enforced. Within this scheme, the Categories dimension introduces a taxonomy for classifying architectural rules in MBSs, while the ACC Scopes dimension identifies three complementary scopes—*Micro-Architecture Conformance Checking*, *Boundary Architecture Checking*, and *Inter-Service Compliance Checking*—that define the enforcement scope of architectural rules in MBSs.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the research methodology. Section 4 reports the execution of the SLR protocol. Section 5 presents and discusses the findings of the review. Section 6 discusses the threats to validity. Finally, Section 7 concludes the paper.

2 Related Secondary Studies

Although we did not identify any secondary study specifically dedicated to ACC in the context of MBSs, several reviews investigate individual activities that support this process. These studies provide valuable insights into architectural representation, architecture recovery, visualization, and degradation. However, none provides a comprehensive characterization of how ACC is performed in MBSs.

Architectural representation constitutes one of the main challenges for ACC in MBSs. Lelovic et al. [26] surveyed architectural languages for representing MBSs, highlighting the difficulty of maintaining consistent architectural models as system complexity increases. Gortney et al. [20] investigated dynamic analysis approaches for visualizing MBSs, showing that the decentralized nature of microservice architectures hinders the construction of a unified architectural view and that runtime-based reconstruction depends on execution coverage. Together, these studies demonstrate that representing the IAr remains a major challenge for ACC.

Architecture recovery is another fundamental activity supporting ACC. Bakhtin et al. [8] surveyed tools for recovering microservice architectures, reporting challenges related to tool availability, maintenance, and the reconstruction of distributed architectures. Schneider et al. [43] evaluated architecture recovery tools and concluded that combining deployment artifacts with source code analysis leads to more reliable architectural models. Together, these studies reinforce that recovering the implemented architecture remains a challenging prerequisite for ACC.

Architectural degradation has also been investigated from complementary perspectives because it frequently results from divergences between PAr and IAr. Existing secondary studies address architectural erosion [28], architectural degradation in open-source systems [7], and architectural and design smells [2, 40]. Although these studies investigate problems closely related to ACC, they primarily focus on degradation symptoms rather than on architectural conformance verification itself.

More recently, Schmid et al. [42] reviewed the application of Large Language Models (LLMs) in software architecture. Their findings indicate that LLMs are increasingly being adopted to support architectural activities; however, Architectural Conformance Checking, particularly in cloud-native and microservice-based systems, remains largely unexplored.

Overall, existing secondary studies investigate isolated activities that support ACC, including architectural representation, architecture recovery, visualization, degradation analysis, and AI-assisted software architecture. Nevertheless, none systematically characterizes ACC itself in the context of MBSs. In particular, previous reviews do not investigate which architectural rules are verified, how planned and implemented architectures are represented, or how conformance checking is actually performed in MBSs.

3 Research Method

This study presents a Systematic Literature Review (SLR) to characterize ACC in MBSs following the Kitchenham et al. guidelines [22]. This section describes the research protocol applied in this review. All artifacts generated during the execution of this Systematic Literature Review are publicly available in [39]. The review was guided by the following research questions (RQs).

RQ1: What categories of architectural rules exist and how they are represented in MBSs? The goal of this RQ is to investigate the types of architectural rules addressed by studies involving ACC in MBSs, categorize them, and identify how such rules are specified and represented. This is important because architectural rules in MBSs may encompass different categories that are specific to the microservice architectural style.

RQ2: How is the implemented architecture recovered and represented? ACC requires somehow obtaining the implemented architecture (IAr) to enable comparison with the architectural rules. Therefore, this question investigates how the architecture of MBS is recovered from implemented systems, including the approaches, tools, and techniques employed for architecture reconstruction. Additionally, it examines how the recovered architectures are modeled and represented.

RQ3: How is conformance checking performed? This question aims to investigate how ACC is carried out in MBS, focusing on the techniques, approaches, and processes used to compare the architectural rules with the IAr. In particular, it examines how conformance is verified, including how violations are detected, the level of automation, and whether the analysis is performed at design-time, runtime, or both.

Search String and Databases

The search string was structured based on two main dimensions: (i) microservices and (ii) ACC. A simplified version of the search string is presented in Listing 1. The complete search string, including all synonyms and database-specific adaptations, is available in [39].

Listing 1: Simplified search string

```
( "microservice architecture" OR "microservices"
  OR "service-oriented architecture" OR SOA
  OR "cloud-native" OR "event-driven architecture" )
AND (
  "architectural conformance" OR "conformance checking"
  OR "architecture compliance" OR "architectural consistency"
  OR "architecture erosion" OR "architecture drift" )
```

Regarding the digital libraries, we have used ACM Digital Library, IEEE Xplore Digital Library, Scopus, and Web of Science. These databases provide broad coverage of peer-reviewed publications, including both journals and conferences.

Inclusion and Exclusion Criteria

The inclusion and exclusion criteria were applied throughout the study selection process, including both the initial screening of studies and the subsequent snowballing stage.

- IC1** : Studies that explicitly investigate ACC in software systems.
- IC2** : Studies that propose, analyze, or evaluate methods, techniques, approaches, tools, or frameworks for ACC.
- IC3** : Empirical studies in which ACC is a primary concern.
- IC4** : Studies addressing ACC in MBS, including cloud-native and event-driven distributed architectures.
- EC1** : Studies that do not address ACC as a central topic.
- EC2** : Studies that discuss software architecture in general without addressing conformance, compliance, or consistency.
- EC3** : Secondary studies.
- EC4** : Non-peer-reviewed publications.
- EC5** : Short papers lacking sufficient methodological details.
- EC6** : Studies not written in English.
- EC7** : Studies without full-text availability.
- EC8** : Theses, dissertations, monographs, or books.
- EC9** : Studies that are clearly misaligned with the research questions, even if they mention ACC-related terms.

IA-Supported Study Selection Process

The study selection process was conducted in two stages: (1) selection of studies retrieved from the digital libraries, and (2) backward and forward snowballing. The snowballing was performed in a single iteration, following the guidelines proposed by Wohlin [52].

Each stage was divided in two steps: (i) an initial screening based on title, abstract, and keywords, and (ii) a full-text assessment of the studies not excluded in the first phase. The selection criteria were consistently applied throughout both phases to determine study relevance. A conservative approach was adopted to minimize the risk of excluding potentially relevant studies, favoring inclusion in cases of uncertainty.

Listing 2: Excerpt of screening prompt

```
You are acting as an independent reviewer in a Systematic Literature
Review (SLR).
Your task is to perform the INITIAL SCREENING of candidate studies
based ONLY on:
- Title
- Abstract
- Keywords
...
For EACH row in the CSV:
1. Independently assess whether the study should be:
   - REJECTED at this stage, or
   - KEPT for the next phase (full-text reading)
2. If the study should be REJECTED, you MUST:
   - Select the MOST APPROPRIATE exclusion criterion from the list
     below
   - Briefly justify your decision in one short sentence
3. If the study CANNOT be confidently rejected based on the
   available metadata,
   classify it as:
   - UNCLASSIFIED (to be forwarded to the next phase)...
```

The initial screening phase was conducted by a researcher, supported by two AI-based agents playing the role of independent reviewers [18]. All retrieved studies were first consolidated and duplicate records were removed. The researcher initially labeled each study as *rejected* or *unclassified* based on the defined inclusion and exclusion criteria.

Subsequently, the same set of studies was independently analyzed by two AI agents (Gemini and Claude), which were instructed to evaluate the studies based solely on title, abstract, and keywords. The agents provided their classification along with the corresponding criteria and justification. An excerpt of the prompt is presented in Listing 2 - the complete one is available in [39].

To support decision-making, studies with conflicting classifications were reassessed by a second researcher. In all cases, the final decision was made by the authors, with AI agents acting only as support for validation. This process was inspired on the guidelines proposed by Felizardo et al. [18]. The use of AI agents was restricted to the initial screening phase and applied consistently in both stages of the selection process. After the initial screening, studies not excluded were included in the full-text assessment phase. In this phase, the evaluation was conducted by the researchers, and disagreements were resolved through discussion.

Quality Assessment

The quality of the selected studies will be assessed after the final selection phase. A checklist will be defined to evaluate the clarity, rigor, and reproducibility of the approaches reported in each study, focusing on aspects directly related to the research questions.

- (1.1) Are the architectural rules clearly defined or described?
- (1.2) Are the architectural rules explicitly specified?
- (1.3) Is the representation or specification of the rules clearly described (e.g., DSL, metrics, models, graphs)?
- (2.1) Is the architecture recovery approach clearly described?
- (2.2) Is the recovery process reproducible?
- (3.1) Is the conformance checking process clearly defined?
- (3.2) Are the criteria for detecting violations clearly specified?
- (3.3) Is the automation level of the ACC process clearly described?

Each study will be evaluated by assigning a score of 0 (not addressed), 0.5 (partially addressed), or 1 (clearly addressed) to each of the eight questions. The overall quality score of each study will be computed as the sum of these scores.

Data Extraction

To answer the RQs, a structured data extraction form was defined and applied to all selected primary studies. The form was organized into three groups corresponding to the research questions. For RQ1, data related to the architectural rules addressed in each study were extracted, including their source, type, and representation. For RQ2, the extraction focused on how the IAr is recovered and represented. For RQ3, the focus was on how ACC is performed, including the techniques used and how violations are identified.

4 SLR Execution

Study Selection

The selection process was conducted in January 2026 and it is summarized in Fig. 1. The search process retrieved a total of 218 studies from the digital libraries. After removing duplicates, 161 studies remained for the initial screening phase.

During the initial screening based on titles, abstracts, and keywords, 115 studies were excluded, resulting in 46 studies for full-text assessment. As previously described, AI agents were used solely as a support mechanism, acting as independent reviewers to provide a second opinion and stimulate discussion among the researchers.

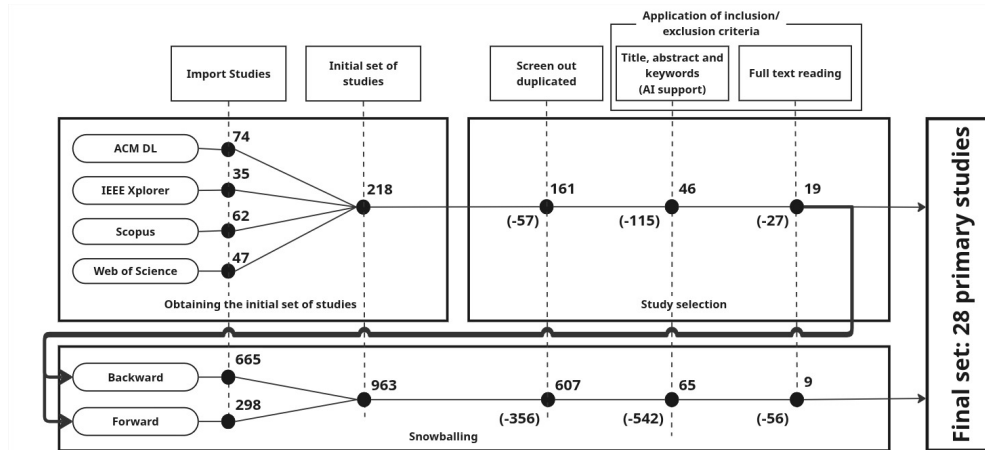


Figure 1: Study selection process.

The primary researcher rejected 121 studies, whereas the AI agents (Claude and Gemini) rejected 71 and 92 studies, respectively. This process resulted in 26 disagreements, which were reassessed by a second researcher to reach the final decision for this phase.

After full-text assessment, 19 studies were selected as primary studies. For the snowballing process, all references cited in the selected studies were collected (backward snowballing), resulting in 665 studies. Additionally, forward snowballing was conducted using Google Scholar, identifying 298 citing studies. This resulted in a total of 963 studies. After removing duplicates, 607 studies remained for the screening phase.

The same screening procedure was applied in the snowballing stage. After title, abstract, and keyword analysis, 65 studies were selected for full-text assessment. Following full-text reading, nine studies were included. Following the process, a total of 28 primary studies were selected and advanced to the quality assessment phase.

Quality Assessment

The quality assessment was conducted on the 28 selected primary studies using the checklist presented in Section 3, concluding the study selection process. As a result, a final set of 28 primary studies was included in this SLR. These studies are presented in Table 1, organized by publication year. To ensure traceability throughout the analysis, each study was assigned a unique identifier (S01–S28), which is used consistently throughout the remainder of this paper.

For RQ1, studies [S01], [S07], [S09], [S24], and [S27] received a score of 1.0 because their architectural rules are explicitly specified using DSLs or formal models. Studies based on Architectural Design Decisions received a score of 0.5 for rule specification, as the rules are described but not formally specified. For RQ2, reproducibility was scored as 1.0 when source code or repositories were publicly available and 0.5 when the study only reported the adopted tool. For RQ3, automation was scored as 0.5 whenever manual intervention was required during the conformance checking process.

Data Extraction

Data extraction was performed on the final set of 28 primary studies according to the extraction protocol. Although all data items were extracted using the predefined extraction form. Extracting architectural rules posed a particular challenge because many primary

studies did not present them explicitly. Instead, architectural rules were often embedded in architectural constraints, design principles, recommendations, architectural patterns, or textual descriptions. In such cases, the researchers identified the underlying architectural rule and reformulated it into an explicit representation while preserving its original semantics. This representation explicitly identifies the architectural elements involved and the required, allowed, or prohibited relationships among them, making the rules suitable for subsequent analysis. This process resulted in a final set of 125 unique rules, which are publicly available in [39].

5 Answers to Research Questions

RQ1 – What categories of architectural rules exist and how they are represented in MBSs?

As a starting point for answering this RQ, it is important to clarify what we consider as an architectural rule. *An architectural rule is an explicit and verifiable statement that prescribes, restricts, or constrains the structure or behavior of a software architecture.* Such rules are typically expressed in a normative form, specifying what architectural elements must, may, or must not do. Examples include statements such as “*Component A must be present in the system*”, “*Layer C1 may only communicate with Layer C3*”, or “*Service A must not access Database B directly*” [24, 50]. A key characteristic of an architectural rule is that it can be objectively checked against an implemented architecture (IAr) to determine whether it is satisfied.

To answer RQ1 comprehensively, we analyzed the set of 125 architectural rules according to four complementary dimensions: (1) categories, (2) how they are represented, (3) the sources which they are derived from, and (4) the scope of ACC in which they are enforced. Fig. 2 summarizes this analytical dimensions that organizes the presentation of the results throughout this section.

Dimension 1 - Category. In order to identify categories for the rules we adopted open coding, an inductive coding technique in which concepts emerge directly from the analyzed data rather than being predefined [41]. Each rule was analyzed individually and assigned one or more conceptual codes representing the architectural concerns captured by the rule. The resulting codes were iteratively refined and subsequently grouped into higher-level categories.

Table 1: Summary of selected studies sorted by year

ID	Authors	Title	Year	Venue
S01	Weinreich; Buchgeher [51]	Automatic reference architecture conformance checking for SOA-Based software systems	2014	WICSA
S02	Zdun; Navarro; Leymann [53]	Ensuring and assessing architecture conformance to microservice decomposition patterns	2017	ICSOC
S03	†Engel et al. [16]	Evaluation of microservice architectures: A metric and tool-based approach	2018	CAISE
S04	†Brogi; Neri; Soldani [9]	Freshening the air in microservices: resolving architectural smells via refactoring	2019	ICSOC
S05	Ntontos et al. [35]	Assessing architecture conformance to coupling-related patterns and practices in microservices	2020	ICSOC
S06	Ntontos et al. [36]	Metrics for assessing architecture conformance to microservice architecture patterns and practices	2020	ICSOC
S07	Araujo et al. [6]	Applying a multi-platform architectural conformance solution in a real-world microservice-based system	2020	SBCARS
S08	Apolinário; de França [13]	Towards a method for monitoring the coupling evolution of microservice-based architectures	2020	SBCARS
S09	Krieger et al. [25]	Monitoring Behavioral Compliance with Architectural Patterns Based on Complex Event Processing	2020	ESOCC
S10	†Soldani et al. [47]	The μ TOSCA toolchain: Mining, analyzing, and refactoring microservice-based architectures	2021	SPE
S11	Ntontos et al. [33]	Evaluating and Improving Microservice Architecture Conformance to Architectural Design Decisions	2021	ICSOC
S12	Ntontos et al. [34]	Semi-automatic Feedback for Improving Architecture Conformance to Microservice Patterns and Practices	2021	ICSA
S13	Apolinário; de França [5]	A method for monitoring the coupling evolution of microservice-based architectures	2021	JBCS
S14	†Farsi et al. [17]	A graph-based solution to deal with cyclic dependencies in microservices architecture	2022	FiCloud
S15	Singjai; Zdun [46]	Conformance Assessment of Architectural Design Decisions on the Mapping of Domain Model Elements to APIs and API Endpoints	2022	ICSA-C
S16	Ntontos et al. [31]	Assessing Architecture Conformance to Security-Related Practices in Infrastructure as Code Based Deployments	2022	SCC
S17	Ntontos et al. [37]	Assessing Architecture Conformance to Coupling-Related Infrastructure-as-Code Best Practices: Metrics and Case Studies	2022	ECSCA
S18	Singjai; Zdun [44]	API Description-Based Conformance Assessment of Architectural Design Decision	2022	SOSE
S19	Singjai; Zdun [45]	Conformance assessment of Architectural Design Decisions on API endpoint designs derived from domain models	2022	JSS
S20	†Daniel et al. [12]	Towards the detection of microservice patterns based on metrics	2023	SEAA
S21	Ntontos et al. [32]	Detecting and Resolving Coupling-Related Infrastructure as Code Based Architecture Smells in Microservice Deployments	2023	CLOUD
S22	†Adams et al. [3]	Evolution and anti-patterns visualized: Microprospect in microservice architecture	2024	ECSCA
S23	†Sun et al. [49]	Detecting inconsistencies in microservice-based systems: An annotation-assisted scenario-oriented approach	2024	TSC
S24	Zhong et al. [55]	DOMICO: Checking conformance between domain models and implementations	2024	SPE
S25	Zdun et al. [54]	Detection Strategies for Microservice Security Tactics	2024	TDSC
S26	Cao et al. [10]	CATMA: Conformance Analysis Tool for Microservice Applications	2024	ICSE-C
S27	†Giallorenzo et al. [19]	A Toolchain for Checking Domain-and Model-Driven Properties of Jolie Microservices	2025	ICSOC
S28	†Abdelfattah et al. [1]	Semantic dependency in microservice architecture	2025	ICSR

Note: Studies marked with † were obtained through snowballing.

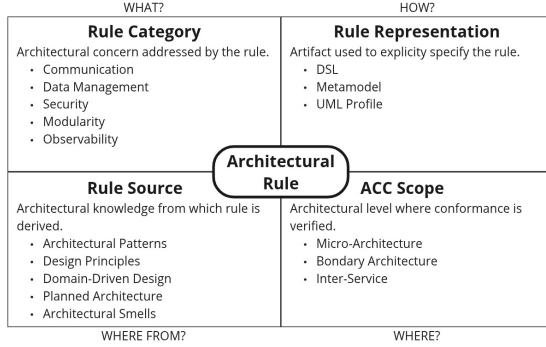


Figure 2: Classification Scheme of Architectural Rules.

The open coding process resulted in 265 conceptual codes, which were iteratively refined and grouped into six high-level categories. Table 2 summarizes the resulting taxonomy, including the categories and subcategories, the number of rules assigned to each subcategory, and the corresponding primary studies. The complete open coding process, along with the consolidated rules and their classifications, is available in our replication package [39].

The *Domain Boundary* category encompasses rules that apply Domain-Driven Design (DDD) principles to ensure business logic defines the architectural boundaries of the software, both internally and in the exposure of service contracts. These rules aim to preserve the semantic integrity of domain models while maintaining alignment between the implementation and the underlying business concepts. The *Tactical DDD Pattern Enforcement* subcategory comprises rules related to the application of DDD patterns, such as *Aggregates*, *Entities*, and *Value Objects*. The *Bounded Context & Model Isolation* subcategory includes rules for isolating *Bounded Contexts* and preventing model leakage across contexts. Finally, the *Domain-API Contract Mapping* subcategory covers rules related to traceability and semantic alignment between APIs, contracts,

domain events, and business models. This category comprises 28 architectural rules identified across 10 primary studies.

The *Inter-Service Communication* category comprises rules that govern communication among microservices, aiming to minimize both structural and temporal coupling while preserving service autonomy. This is the category with the largest number of identified architectural rules. The *Asynchronous & Non-Blocking Interaction* subcategory includes rules related to event-driven communication, the use of messaging mechanisms, and temporal decoupling between services. The *Synchronous Interaction & API Gateway Governance* subcategory encompasses rules governing synchronous interactions, service orchestration, and the centralization of communications through API Gateways. In turn, the *Coupling & Dependency Management* subcategory covers rules related to dependency control, the prevention of cyclic dependencies, and the governance of microservice granularity. Overall, this category comprises 37 rules identified across 18 primary studies, highlighting that communication management and coupling control constitute the primary architectural concern addressed by the analyzed studies.

Data decentralization is one of the fundamental principles of microservice architecture. In this context, the *Data Management* category encompasses rules that embody the principle of data sovereignty by ensuring persistence isolation and preventing inappropriate database sharing across services. The *Database-per-Microservice Ownership* subcategory comprises rules related to the exclusive ownership of a database by a single microservice. In turn, the *Persistence Layer Purity & Encapsulation* subcategory includes rules aimed at preventing persistence logic from leaking into the domain layer, enforcing the isolation of *Data Access Objects* (DAOs) and *Repositories*, and promoting the encapsulation of the persistence layer. Overall, this category comprises 12 architectural rules identified across 9 primary studies.

Resilience and observability are inherent concerns in applications distributed across multiple processing nodes. These concerns

Table 2: Taxonomy of Architectural Rules Identified through Open Coding [41]cma

Category / Subcategory	Examples of Architectural Rules	# Rules	Studies
1. Domain Boundary			
Bounded Context & Model Isolation	Microservice APIs shall not leak internal domain models directly to external clients.[S15; S18]	4	[S15], [S18], [S19], [S24]
Domain-API Contract Mapping	Microservices should not be organized around business entities.[S03]	7	[S03], [S15], [S18], [S19], [S27]
Tactical DDD Pattern Enforcement	External references to an Aggregate shall only capture the unique identity (ID) of its Aggregate Root.[S15]	17	[S01], [S06], [S11], [S15], [S19], [S20], [S24], [S27]
2. Inter-Service Communication			
Asynchronous & Non-Blocking Interaction	Inter-service write operations shall prioritize asynchronous message brokers or event-driven interaction styles over blocking synchronous calls.[S24]	6	[S01], [S06], [S11], [S20], [S24], [S27]
Coupling & Dependency Management	Microservices shall not have cyclic dependencies.[S14;S22]	24	[S01], [S03], [S04], [S05], [S10], [S12], [S13], [S14], [S16], [S17], [S20], [S21], [S22], [S24], [S27]
Synchronous Interaction & API Gateway Governance	External clients shall access system capabilities through a centralized API Gateway rather than contacting internal microservices directly.[S24]	7	[S03], [S09], [S24]
3. Data Management			
Database-per-Microservice Ownership	Each microservice shall maintain absolute ownership over its private database schema, prohibiting external services from bypassing its API.[S19]	2	[S19], [S22]
Persistence Layer Purity & Encapsulation	Repositories shall hide database implementation details and map infrastructure data structures directly to clean domain models.[S04;S10]	10	[S01], [S04], [S09], [S10], [S15], [S24], [S27]
4. Operational Resilience & Observability			
Distributed Telemetry & Observability	Microservices shall output standardized telemetry and operational logs to centralized monitoring infrastructure.[S24]	10	[S03], [S06], [S11], [S16], [S19], [S20], [S24]
Fault Tolerance & Self-Healing	Remote service calls shall use circuit breakers and fallbacks to prevent cascading system failures.[S01]	10	[S01], [S04], [S09], [S10], [S16], [S19], [S20], [S24], [S27]
5. Modularity & Deployment Autonomy			
Internal Modularity & Package Separation	Each architectural module shall expose its features through a restricted public interface and completely encapsulate its internal implementation details.[S25]	6	[S02], [S20], [S24], [S25], [S27]
Independent Deployment & Lifecycle	Microservices shall be packaged and run inside isolated infrastructure containers to guarantee strict resource isolation.[S01]	10	[S01], [S03], [S05], [S12], [S17], [S19], [S20], [S21], [S22]
6. Security			
Identity & Access Management (IAM)	The system shall use SSL/TLS-based or token-based authentication for identity verification and communication encryption.[S16]	6	[S02], [S15], [S16], [S20], [S25]
Network Security & Traffic Protection	Server nodes shall restrict outbound network traffic to prevent external attacks and data leakage.[S16]	6	[S05], [S12], [S14], [S16]

are captured by the *Operational Resilience & Observability* category, which encompasses rules related to the operational behavior of microservices in distributed environments, including mechanisms for fault tolerance, monitoring, and observability. The *Fault Tolerance & Self-Healing* subcategory comprises rules concerning the application of architectural patterns such as *Circuit Breaker* and *Rate Limiting*, as well as mechanisms for preventing cascading failures and enabling automatic service recovery. In turn, the *Distributed Telemetry & Observability* subcategory includes rules related to distributed tracing, centralized logging, and the monitoring of infrastructure and system resources. Overall, this category comprises 20 rules identified across 12 primary studies.

The *Modularity & Deployment Autonomy* category encompasses rules that promote the modularity of architectural components and their autonomy throughout the software lifecycle, enabling independent deployment and evolution. The *Independent Deployment & Lifecycle* subcategory comprises rules related to deployment independence, microservice instance autonomy, and the isolation provided by containers. In turn, the *Internal Modularity & Package Separation* subcategory includes rules concerned with internal modularity, separation of concerns, encapsulation, and code-level cohesion. Overall, this category comprises 16 architectural rules identified across 13 primary studies.

Finally, the *Security* category encompasses cross-cutting rules aimed at protecting the distributed ecosystem, covering authentication, authorization, secure communication, and network traffic governance. The *Identity & Access Management* subcategory comprises rules related to token-based authentication, authorization, and *Single Sign-On* (SSO) mechanisms. In turn, the *Network Security & Traffic Protection* subcategory includes rules concerned with

protecting communications through encryption, controlling network traffic, and preventing data leakage. Overall, this category comprises 12 architectural rules identified across 8 primary studies.

The resulting taxonomy suggests that ACC in MBS goes beyond the traditional view of comparing PAr with the IAr. The architectural rules predominantly operate at a higher level of abstraction, focusing on preserving architectural properties and enforcing design principles and architectural decisions throughout system evolution. Rather than primarily specifying explicit structural constraints, most studies define rules intended to promote desirable architectural properties, such as loose coupling, deployment independence, data sovereignty, resilience, and secure communication. Furthermore, the identified categories closely reflect the fundamental principles that underpin the microservice architectural style.

Most of the identified rules seek to preserve service autonomy through constraints related to inter-service communication, decentralized data ownership, and the isolation of Bounded Contexts. In contrast, a considerably smaller number of rules address traditional implementation-level constraints, such as dependencies among packages, components, or layers, which are commonly found in ACC approaches targeting monolithic systems. This finding indicates that, in the context of microservices, ACC is no longer primarily concerned with verifying structural constraints, but increasingly incorporates the verification of architectural properties related to service autonomy, distributed communication, and the preservation of quality attributes.

Dimension 2 - Representation. Still within RQ1, we analyzed how architectural rules are represented in the retrieved studies. In this work, we define *an architectural rule representation as the artifact that explicitly specifies an architectural rule or constraint,*

allowing it to be understood and subsequently verified. Table 3 summarizes the studies that explicitly represent architectural rules, the artifact used for this representation, and the purpose of the representation. We observe that only a small subset of the primary studies ([S01;S07;S09;S23;S24;S27]) explicitly represents architectural rules through dedicated artifacts, such as DSLs, metamodels, and UML profiles. In most studies, however, rules are not explicitly represented; instead, they are inferred from their source, that is, from higher-level artifacts such as architectural patterns, design principles, and architectural decisions.

Table 3: Architectural Rule Representations

Study	Representation	Purpose
[S01]	LISA (DSL)	Architectural dependency rules among software components.
[S07]	DCL+ (DSL)	Communication constraints between microservices.
[S09]	Esper EPL (DSL)	Runtime behavioral compliance rules.
[S23]	MSDM (Metamodel + DSL)	Expected microservice architecture and architectural rules.
[S24]	UML Profile Model	DDD concepts and architectural rules using UML stereotypes.
[S27]	LEMMA (DSL)	Microservice architecture and DDD concepts for ACC.

Among the studies that propose an explicit representation of architectural rules, [S01], [S07], [S24], and [S27] primarily focus on specifying structural constraints related to the organization of microservices. Studies [S01] and [S07] employ DSLs to describe constraints on dependencies and relationships among microservices, whereas [S24] and [S27] adopt models based on DDD concepts, representing elements such as *Bounded Contexts*, *Entities*, and *Aggregates*. In contrast, studies [S09] and [S23] broaden the scope of rule representation by incorporating behavioral aspects. Study [S09] uses a DSL to specify the expected runtime behavior of the system, whereas [S23] proposes the Microservice Description Model (MSDM), a metamodel accompanied by a DSL capable of representing both the expected structure and behavior of a MBSs. These findings indicate that, although most explicit representations remain focused on specifying architectural structure, some studies have begun to incorporate behavioral aspects, reflecting the need to verify dynamic properties inherent to microservice architectures.

Dimension 3 - Source. By analyzing the identified architectural rules and their corresponding representations, we observed that they are derived from different sources of architectural knowledge. In this study, we define the source of an architectural rule as the architectural knowledge that underlies its definition. Such a source may correspond, for example, to architectural patterns, design principles, DDD concepts, or architectural models. Therefore, we observed that architectural rules do not originate exclusively from the PAr or from structural constraints among components, as has traditionally been the case in ACC. Instead, they are also derived from higher-level architectural knowledge that expresses principles, patterns, and best practices for developing MBSs.

Among the sources directly related to architectural specification, we identified reference architectures ([S01]) and the planned architecture ([S07], [S09], [S23], [S24], and [S27]). However, most architectural rules are derived from conceptual sources, such as Architectural Patterns ([S02], [S09], and [S20]), design principles ([S03], [S08], and [S13]), Architectural Smells and Anti-patterns ([S04], [S10], [S14], and [S22]), and DDD concepts ([S24] and [S27]).

Some studies ([S05], [S06], [S11], [S12], [S15], [S16], [S17], [S18], [S19], [S21], and [S25]) identify Architectural Design Decisions

(ADDs) as the source of architectural rules. However, our analysis indicates that the documented ADDs are grounded in previously established architectural knowledge, particularly architectural patterns. For example, decisions concerning the adoption of the *API Gateway*, *Messaging*, and *Database per Service* patterns are directly derived from these patterns, while ADDs are used to document the decision to adopt them in a specific context. Consequently, in this review, we classified these studies according to the architectural knowledge source that underlies each rule, rather than according to the mechanism used to document the architectural decision.

Dimension 4 - Scope of Architectural Enforcement. As final step of the analysis, we have observed the rules differ not only in the categories and representations, but also in the scope in which they are enforced. Therefore, we classified each rule in three *ACC scopes*:

The first scope, referred to as *Micro-Architecture Conformance Checking*, corresponds to the traditional scope of ACC, in which conformance is verified within an individual microservice. This scope includes rules related to dependencies among packages, layers, classes, and method invocations, as well as naming conventions and other structural and semantic constraints.

The second scope, referred to as *Boundary Architecture Checking*, encompasses rules related to the architectural boundaries of an application, regardless of whether it is implemented as a microservice or as a monolithic system. In this scope, conformance is verified over the artifacts exposed by the application, such as APIs, service contracts, events, messages, and other communication interfaces. Unlike *Micro-Architecture Conformance Checking*, which focuses on the application's internal implementation, this scope is concerned with verifying whether the application's public interface complies with the defined constraints, regardless of its internal architecture.

In turn, the *Inter-Service Compliance Checking* scope shifts the focus of analysis to the relationships among applications. Rules in this scope verify architectural properties that depend on the interaction among multiple services or systems, including communication topologies, synchronous and asynchronous protocols, service dependencies, integration strategies, and compliance with architectural principles and patterns specific to distributed systems.

Table 4 presents the distribution of the 125 architectural rules across the identified ACC scopes, reporting both the absolute number and the percentage of rules classified in each scope. The results show that most architectural rules (47.2%) belong to the *Inter-Service Compliance Checking* scope, indicating that the primary architectural concerns in MBS are associated with interactions among services, including communication, deployment, resilience, scalability, observability, and operational security. When combined with the rules classified under *Boundary Architecture Checking*, 68.8% of the rules are related either to application interfaces or to interactions among services. This finding indicates ACC shifts its focus from verifying the internal structure of applications toward distributed architectural concerns, reflecting the increasing importance of service interfaces, inter-service communication, and service autonomy in preserving architectural conformance.

The analysis performed in RQ1 shows that architectural rules in MBSs are considerably more heterogeneous than those traditionally found in ACC for monolithic systems. Rather than being limited to structural constraints defined over architectural models, the identified rules encompass multiple architectural concerns, originate

Table 4: Classification of Rules Across ACC Scopes

ACC Scope	Scope of Verification	# Rules	Percentage
Micro-Architecture Conformance Checking	Internal implementation of the application, including packages, layers, classes, methods, and code-level constraints.	39	31.2%
Boundary Architecture Checking	External interfaces of an application, including APIs, events, messages, and service contracts, regardless of whether the application is implemented as a monolith or a microservice.	27	21.6%
Inter-Service Compliance Checking	Architectural relationships among services, including communication, dependencies, and distributed architectural principles.	59	47.2%
Total	—	125	100.0%

from different sources of architectural knowledge, are explicitly represented only in a limited number of studies, and are verified at distinct architectural scopes. Collectively, these findings suggest that architectural rules can be systematically characterized through four complementary perspectives: what architectural concern they address, how they are represented, from where they originate, and where architectural conformance is verified.

RQ2 — How is the IAr recovered/represented?

So that an ACC process can be conducted, the architectural rules must be checked against an artifact that retains how the system is currently implemented, known as *implemented architecture* (IAr). To answer this RQ we have analyzed how IAr is recovered and represented, considering recovery strategies, the degree of automation, and architectural artifacts used during the recovery process.

Initially we have classified the studies according to the recovery strategy: static and dynamic, as can be seen in Table 5. Static analysis is based on inspecting structural system artifacts, such as source code, API descriptions, and infrastructure scripts, whereas dynamic analysis relies on artifacts produced during system execution, such as logs, traces, events, and service-to-service communication data.

Table 5 also indicates the level of automation of the architectural recovery process and the artifacts analyzed in each case. It can be observed that some studies combine static and dynamic analysis techniques to obtain a more comprehensive view of the IAr, thus characterizing a hybrid architecture recovery approach. These studies are marked with the symbol ‘*’.

Table 5: Architecture Recovery Approaches Identified

Recovery Strategy: Static/- Dynamic	Automation Level	Analyzed Artifact
Static Analysis		
[S01], [S26]*	Automatic	Source code and configuration files
[S02], [S22], [S23]*, [S27]	Automatic	Source code
[S07], [S11], [S12], [S15], [S25]	Semi	Source code
[S05], [S06], [S20]	Manual	Source code
[S10]*	Automatic	Kubernetes manifests
[S16], [S17], [S21]	Semi	IaC scripts (Ansible/Terraform)
[S18]	Automatic	API specifications (OpenAPI/Swagger)
[S22], [S24], [S28]	Automatic	Source code
Dynamic Analysis		
[S03]	Semi	Communication data (logs/traces)
[S08], [S13]	Semi	Distributed tracing (Istio/Jaeger)
[S09]	Automatic	Runtime events
[S10]*	Automatic	Network packet sniffing
[S23]*	Automatic	Observed behavior through distributed tracing
[S26]*	Automatic	HTTP event logs collected from Kubernetes

A strong tendency toward automation in the architectural recovery process can be observed. Approximately 46% of the studies perform architecture recovery in a fully automated manner, while

another 42% adopt semi-automated approaches, resulting in nearly 89% of the retrieved studies employing some level of automation.

Studies that fully automate the recovery process rely on tools capable of extracting architectural information directly from artifacts such as source code, deployment specifications, and network traffic. In contrast, studies adopting semi-automated approaches automate the collection of raw data, such as logs, traces, and code-derived information, but still require human involvement in activities such as refining, grouping, or correcting the recovered architectural model.

For the studies in which architecture recovery is performed manually, this procedure is primarily used for research, validation, or experimental evaluation purposes.

The most frequently used tool among the analyzed studies is *CodeableModels*, which is adopted for ten studies. The tool enables the precise specification of metamodels, models, and model instances directly in the source code, while also supporting the generation of PlantUML visualizations [35].

Studies [S04] and [S10] propose a dedicated tool named *μMINER*, which is based on the *OASIS TOSCA* standard for architectural modeling and analysis. The remaining studies rely on tools specifically developed for their own approaches. The diversity of identified solutions suggests that architectural recovery for microservices has not yet reached a consolidated or widely adopted tooling ecosystem.

Several studies aim to recover relationships and dependencies among microservices through source code analysis, using implementation annotations to identify architectural elements. For instance, studies [S07], [S23], and [S28] use the *@FeignClient* annotation to identify synchronous communication among services. In addition, [S23] and [S28] employ the *@Controller*, *@Service*, and *@Repository* annotations to extract structural information from the application. Study [S24] uses the *@Entity* annotation to identify instances of the DDD *Entity* pattern. Similarly, [S27] employs the *@aggregate*, *@entity*, *@valueObject*, and *@ctx* annotations to document and recover architectural concepts related to DDD.

However, these approaches are limited by their dependence on specific technologies, frameworks, or conventions. For instance, [S07] identifies microservice communication via Java/Spring *@FeignClient* annotations. However, newer framework versions allow service-to-service communication through alternative mechanisms without this annotation, compromising the completeness and accuracy of the architectural recovery.

Since microservice architecture consists of distributed, independently deployable components that communicate over the network, several approaches rely on deployment and runtime artifacts to recover the IAr. For example, studies [S10] and [S18] use Kubernetes manifests and API specifications to identify relationships among microservices, whereas [S16], [S17], and [S21] analyze Infrastructure as Code (IaC) deployment scripts to recover dependencies and relationships among architectural components.

Compared with traditional ACC, we observe that the IAr is no longer concentrated in the source code. In MBSs, architectural information is distributed across multiple development, deployment, and runtime artifacts. Consequently, the architecture recovery process combines multiple sources of information, integrating evidence obtained from both static and dynamic analysis techniques. This finding establishes a parallel with the results of RQ1: whereas architectural rules are derived from multiple sources of architectural

knowledge, the IAr is recovered from multiple architectural artifacts. In both cases, the distributed nature of microservices broadens the scope of ACC, which no longer relies on a single artifact but instead integrates multiple sources of information to represent both the architectural knowledge and its implementation.

Unlike RQ1, which investigated how architectural rules are explicitly represented, this stage analyzes how the IAr, once recovered, is represented to support the ACC process. Therefore, the notion of representation discussed in this section refers to the model used to describe the IAr rather than the artifact used to specify rules.

Regarding the representation of the IAr, a predominance of graph-based models can be observed. Studies [S03], [S05], [S06], [S08], [S11], [S12], [S13], [S14], [S15], [S16], [S17], [S19], [S21] and [S25] use dependency graphs to represent synchronous and asynchronous interactions among microservices. Meanwhile, studies [S22], [S23], and [S28] employ *Component Call Graphs* (CCGs) to capture execution flows across internal architectural layers as well as remote service invocations.

Another common form of representation is based on architectural models and metamodels. Studies [S04] and [S10], for example, adopt μ TOSCA, a specialization of the OASIS TOSCA standard that represents the architecture as a typed topology graph. Study [S01] employs the *LISA Architecture Meta-model*, which provides object-oriented abstractions for low-level elements, such as classes and interfaces, and design abstractions for higher-level concepts, such as modules, components, and services. In turn, study [S23] proposes the Microservice Description Model (MSDM), which captures fine-grained architectural information, including API specifications, internal component structure, and resource dependencies.

Still within the context of model-based representations, studies [S24] and [S27] represent the architecture using DDD concepts, such as *Bounded Contexts*, *Aggregates*, *Entities*, and *Value Objects*, as abstractions to describe the structure of microservices.

Although graphs and metamodels are the predominant forms of representing architectural dependencies, some studies adopt alternative strategies. For example, study [S26] uses Data Flow Diagrams (DFDs) to visualize the flow of information among microservices and identify potential security vulnerabilities. In contrast, study [S28] employs a semantic dependency matrix, represented as a heatmap, to quantify the strength of dependencies among microservices based on shared functionalities.

The results show that the representation of the IAr favors models that explicitly capture dependencies and communication flows among services. The predominance of graph-based representations indicates the literature prioritizes modeling the connectivity of distributed architectures, which is essential for verifying properties such as coupling, service dependencies, and inter-service communication. In contrast, only a few studies ([S23] and [S26]) employ metamodels and data flow to support specific analysis objectives.

In summary, RQ2 shows that recovering the IAr in MBSs differs substantially from traditional monolithic ACC. Rather than relying solely on source code, it reconstructs the architecture from multiple distributed artifacts across development, deployment, and runtime information. The result is predominantly represented as dependency and communication graphs, capturing the distributed nature of microservices to support ACC.

RQ3 — How is conformance checking performed?

Considering the traditional ACC process, which identifies architectural violations by comparing the PAr with the IAr, only five (17%) of the retrieved studies explicitly adopt this approach.

From a structural conformance perspective, study [S07] uses DCL+ to formally specify the constraints of the PAr and compare them with the communications and architectural structure extracted from the source code of the microservices. Study [S24] proposes DOMICO (Domain Model & Implementation Conformance), which uses the domain model, based on DDD, as the representation of the PAr and compares it with its implementation to identify *architectural drifts*. Along the same line, study [S27] presents the Jolie Checker Toolchain (JCT), whose goal is to keep the implementation consistent with the architectural model by verifying whether the evolution of the source code remains aligned with the intended design. Also from a structural perspective, study [S01] proposes an approach for verifying conformance between the implementation and a *Reference Architecture* for SOA-based systems.

From a behavioral perspective, study [S23] compares the planned behavior with the behavior observed at runtime. To do this, it uses the MSDM as the specification of the PAr and compares the expected behavior with the behavior obtained through distributed tracing.

In contrast, a predominance of studies employ ACC to assess, monitor, and improve architectural quality. This finding indicates that, in the context of MBSs, ACC has evolved from comparing planned and implemented architectures toward the continuous assessment of architectural quality based on architectural indicators, quality metrics, and the verification of compliance with architectural principles, patterns, and design decisions. In these studies, architectural conformance is assessed through the definition of metrics, the identification of *architectural smells*, and the evaluation of adherence to best practices, using these indicators to measure the architectural integrity and overall health of the system.

The use of metrics is recurrent among the retrieved studies. In this approach, the IAr is first analyzed to extract quantitative indicators that characterize specific architectural properties. These indicators are then compared against predefined thresholds, reference values, or evaluation criteria, allowing the architectural conformance level of the system to be inferred. For example, study [S03] defines metrics to identify critical architectural issues, including the number of synchronous and asynchronous interfaces, the distribution of service calls to assess the Single Responsibility Principle, and the number of dependencies to measure coupling. Focusing on coupling, studies [S08] and [S13] propose metrics such as *Absolute Service Importance*, *Absolute Dependency*, and *Service Coupling Factor*. Studies [S20] and [S28] also employ metrics related to coupling and cohesion. Study [S02], in turn, evaluates the quality of microservice architectures based on violations of decomposition patterns. Studies [S05], [S06], [S11], and [S12] use metrics to assess compliance with ADDs derived from architectural patterns. In addition, studies [S16], [S17], [S21], and [S25] propose metrics targeting IaC and security-related aspects.

Some studies interpret architectural conformance as the absence of anomalies that compromise specific quality attributes, relying on the detection of *architectural smells* and the application of refactoring strategies. Studies [S04] and [S10] present approaches for

identifying *smells* that violate principles such as horizontal scalability, fault isolation, and decentralization. Beyond detecting these anomalies, study [S10] also evaluates how eliminating them through refactoring improves system performance and maintainability. Similarly, studies [S11] and [S12] use metrics to detect and resolve violations related to ADDs, particularly those associated with coupling among microservices. Along the same line, study [S21] identifies violations in IaC models to preserve principles such as deployment independence and service scalability.

In summary, RQ3 reveals an important shift in how ACC is performed in MBSs. Although a subset of the studies [S01;S07;S23;S24;S27] still adopts the traditional process of comparing PAr against IAr, considering both structural and behavioral perspectives, most of the literature performs ACC through the evaluation of architectural properties, using metrics, the detection of smells, and the verification of compliance with architectural patterns and design decisions. Together with the findings of RQ1 and RQ2, these results indicate that ACC has evolved from an approach predominantly focused on verifying structural consistency between planned and implemented architectures to one centered on the assessment of architectural quality. Consequently, ACC expands its role from detecting architectural violations to supporting the preservation of architectural quality in MBSs.

6 Threats to Validity

Construct validity threats relate to the interpretation and classification of architectural rules, given that primary studies employ heterogeneous terminology and the taxonomy emerged inductively via *open coding*. To mitigate this risk, explicit classification criteria and a structured extraction form were adopted, with iterative refinement of categories until conceptual consistency was achieved.

Internal validity threats involve potential biases in study selection and data extraction. Although AI-based tools supported the study selection process, they acted solely as an aid; all inclusion, exclusion, and data extraction decisions were manually reviewed and validated by the authors according to the predefined criteria.

External validity threats limit the generalizability of the findings to the context of ACC for MBS. To maximize coverage and mitigate the omission of relevant approaches, the search across multiple libraries was complemented by forward and backward *snowballing*.

Conclusion validity threats stem from the qualitative and interpretative nature of the coding, which could allow for alternative categorizations. This factor was minimized by grounding all classifications in explicit evidence extracted from the primary studies and iteratively refined.

7 Conclusion

This work presents a Systematic Review on the employment of Architectural Conformance Checking (ACC) for Microservice-Based Systems (MBS). After answering three research questions we have observed important points.

One of them is that the nature of the architectural rules employed in ACC has evolved significantly in the context of MBSs. Whereas traditional ACC predominantly focuses on structural constraints, our findings show that, in MBSs, rules are primarily concerned with preserving architectural properties such as service

autonomy ([S02]), loose coupling([S05];[S08]), inter-service communication([S07]), and resilience([S09]).

Another important finding concerns the way architectural knowledge is represented. Although architectural rules have become increasingly high-level, only a small number of studies (17%) propose explicit representations based on DSLs or architectural models. In contrast, IAr are predominantly represented as dependency or communication graphs (60%), reflecting the need to capture the communication among services. Furthermore, recovering the IAr is no longer limited to source code analysis, but requires combining multiple artifacts, including source code([S07];[S24]), deployment descriptors([S21]), IaC([S17]), API specifications([S18];[S19]), and configuration files([S25]).

Taken together, these findings indicate a fundamental shift in the ACC paradigm. Rather than focusing primarily on comparing PAr and IAr, ACC in MBSs increasingly relies on recovering distributed architectural information and evaluating architectural properties through metrics, architectural principles, and high-level rules. Consequently, ACC becomes progressively associated with assessing architectural quality attributes such as service autonomy, communication, coupling, resilience, and security.

This work contributes to the literature by proposing a four-dimension classification scheme for architectural rules: categories (what), representations (how), architectural knowledge sources (where from), and ACC scope (where). The first dimension capture in six categories **what** rule is intended to represent, generating a *taxonomy* for the rules. The second classifies rules according to **how** they are represented. The third classifies rules according to their origin. The fourth categorize them considering the scopes in which they can be applied: *Micro-Architecture Conformance Checking*, *Boundary Architecture Checking*, and *Inter-Service Compliance Checking*—that define where rules are enforced in MBSs.

The results also highlight relevant opportunities for future research. There is a clear need for approaches that formally represent high-level architectural rules, bridging the gap between architectural principles and their automated verification. Similarly, there is still no consensus on an IAr representation capable of integrating information from multiple distributed artifacts, such as source code, infrastructure, configurations, APIs, and runtime data. In this context, future research should focus on developing integrated ACC approaches capable of combining multiple information sources, representing architectural rules at higher abstraction levels, and continuously verifying the architectural properties of MBSs.

Artifact Availability

To ensure reproducibility, our replication package—including the raw extraction sheets, open-coding process, and the author’s version of the paper—is permanently archived at <https://doi.org/10.5281/zenodo.22969089>.

Acknowledgements

This work was supported by the National Council for Scientific and Technological Development (CNPq), Brazil (grant no. 406941/2025-4), and Universidad Católica del Norte (Chile) through the project *Identification and Reconciliation of Architectural Violations in Pub-Sub Systems* (Resolution VRIDT No. 224/2024).

References

- [1] Amr S. Abdelfattah, Kari E. Cordes, Austin Medina, and Tomas Cerny. 2025. Semantic Dependency in Microservice Architecture. In *2025 IEEE/ACM 22nd International Conference on Software and Systems Reuse (ICSR)*. 44–54. doi:10.1109/ICSR66718.2025.00011
- [2] Amjad AbuHassan, Mohammad Alshayeb, and Lahouari Ghouti. 2021. Software smell detection techniques: A systematic literature review. *Journal of Software: Evolution and Process* 33, 3 (2021), e2320. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.2320 doi:10.1002/smr.2320 e2320 JSME-19-0205.R3.
- [3] Lauren Adams, Amr S. Abdelfattah, Md Showkat Hossain Chy, Samantha Perry, Patrick Harris, Tomas Cerny, Dario Amoroso d'Aragona, and Davide Taibi. 2024. Evolution and Anti-patterns Visualized: MicroProspect in Microservice Architecture. In *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*, Bedir Tekinerdoğan, Romina Spalazzese, Hasan Sözer, Silvia Bonfanti, and Danny Weyns (Eds.). Springer Nature Switzerland, Cham, 309–325.
- [4] Nour Ali, Sean Baker, Ross O'Crowley, Sebastian Herold, and Jim Buckley. 2018. Erratum to: Architecture consistency: State of the practice, challenges and requirements. *Empirical Software Engineering* 23, 6 (2018), 1868–1869. doi:10.1007/s10664-017-9542-0
- [5] Daniel R.F. Apolinário and Breno B.N. de França. 2021. A method for monitoring the coupling evolution of microservice-based architectures. *Journal of the Brazilian Computer Society* 27, 1 (2021), 17. doi:10.1186/s13173-021-00120-y
- [6] Elena A. Araujo, Álvaro M. Espindola, Vinicius Cardoso Garcia, and Ricardo Terra. 2020. Applying a Multi-platform Architectural Conformance Solution in a Real-world Microservice-based System. In *Proceedings of the 14th Brazilian Symposium on Software Components, Architectures, and Reuse (Natal, Brazil) (SBCARS '20)*. Association for Computing Machinery, New York, NY, USA, 41–50. doi:10.1145/3425269.3425270
- [7] Ahmed Baabad, Hazura Binti Zulzalil, Sa'adah Hassan, and Salmi Binti Baharom. 2020. Software Architecture Degradation in Open Source Software: A Systematic Literature Review. *IEEE Access* 8 (2020), 173681–173709. doi:10.1109/ACCESS.2020.3024671
- [8] Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, and Davide Taibi. 2024. Tools Reconstructing Microservice Architecture: A Systematic Mapping Study. In *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium*, Bedir Tekinerdoğan, Romina Spalazzese, Hasan Sözer, Silvia Bonfanti, and Danny Weyns (Eds.). Springer Nature Switzerland, Cham, 3–18.
- [9] Antonio Brogi, Davide Neri, and Jacopo Soldani. 2020. Freshening the Air in Microservices: Resolving Architectural Smells via Refactoring. In *Service-Oriented Computing – ICSOC 2019 Workshops*, Sami Yangui, Athman Bouguettaya, Xiao Xue, Noura Faci, Walid Gaaloul, Qi Yu, Zhangbing Zhou, Nathalie Hernandez, and Elisa Y. Nakagawa (Eds.). Springer International Publishing, Cham, 17–29.
- [10] Clinton Cao, Simon Schneider, Nicolas E. Diaz Ferreyra, Sisco Verwer, Annibale Panichella, and Riccardo Scandariato. 2024. CATMA: Conformance Analysis Tool For Microservice Applications. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (Lisbon, Portugal) (ICSE-Companion '24)*. Association for Computing Machinery, New York, NY, USA, 59–63. doi:10.1145/3639478.3640022
- [11] Andrea Caracciolo, Mircea Filip Lungu, and Oscar Nierstrasz. 2015. A unified approach to architecture conformance checking. In *2015 12th Working IEEE/IFIP Conference on Software Architecture*. IEEE, 41–50.
- [12] João Daniel, Eduardo Guerra, Thatiane Rosa, and Alfredo Goldman. 2023. Towards the Detection of Microservice Patterns Based on Metrics. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 132–139. doi:10.1109/SEAA60479.2023.00029
- [13] Daniel Rodrigo de Freitas Apolinário and Breno Bernard Nicolau de França. 2020. Towards a method for monitoring the coupling evolution of microservice-based architectures. In *Proceedings of the 14th Brazilian Symposium on Software Components, Architectures, and Reuse (Natal, Brazil) (SBCARS '20)*. Association for Computing Machinery, New York, NY, USA, 71–80. doi:10.1145/3425269.3425273
- [14] André de S. Landi, Fernando Chagas, Bruno M. Santos, Renato S. Costa, Rafael Durelli, Ricardo Terra, and Valter V. De Camargo. 2017. Supporting the Specification and Serialization of Planned Architectures in Architecture-Driven Modernization Context. In *2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC)*, Vol. 1. 327–336. doi:10.1109/COMPSAC.2017.225
- [15] André de S. Landi, Daniel San Martin, Bruno M. Santos, Warteruzannan S. Cunha, Rafael S. Durelli, and Valter V. Camargo. 2022. Architectural conformance checking for KDM-represented systems. *Journal of Systems and Software* 183 (2022), 111116. doi:10.1016/j.jss.2021.111116
- [16] Thomas Engel, Melanie Langermeier, Bernhard Bauer, and Alexander Hofmann. 2018. Evaluation of Microservice Architectures: A Metric and Tool-Based Approach. In *Information Systems in the Big Data Era*, Jan Mendling and Haralambos Mouratidis (Eds.). Springer International Publishing, Cham, 74–89.
- [17] Hassan Farsi, Driss Allaki, Abdeslam En-Nouaary, and Mohamed Dahchour. 2022. A Graph-based Solution to Deal with Cyclic Dependencies in Microservices Architecture. In *2022 9th International Conference on Future Internet of Things and Cloud (FiCloud)*. 254–259. doi:10.1109/FiCloud57274.2022.00042
- [18] Katia Romero Felizardo, Márcia Sampaio Lima, Anderson Deizepe, Tayana Uchôa Conte, and Igor Steinmacher. 2024. ChatGPT application in Systematic Literature Reviews in Software Engineering: an evaluation of its accuracy to support the selection activity. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (Barcelona, Spain) (ESEM '24)*. Association for Computing Machinery, New York, NY, USA, 25–36. doi:10.1145/3674805.3686666
- [19] Saverio Giallorenzo, Fabrizio Montesi, Marco Peressotti, Florian Rademacher, Sabine Sachweh, and Philip Wizenty. 2025. A Toolchain for Checking Domain- and Model-Driven Properties of Jolie Microservices. In *Service-Oriented Computing*, Walid Gaaloul, Michael Sheng, Qi Yu, and Sami Yangui (Eds.). Springer Nature Singapore, Singapore, 161–175.
- [20] Mia E. Gortney, Patrick E. Harris, Tomas Cerny, Abdullah Al Maruf, Miroslav Bures, Davide Taibi, and Pavel Tisnovsky. 2022. Visualizing Microservice Architecture in the Dynamic Perspective: A Systematic Mapping Study. *IEEE Access* 10 (2022), 119999–120012. doi:10.1109/ACCESS.2022.3221130
- [21] Lorin Hochstein and Mikael Lindvall. 2005. Combating architectural degeneration: a survey. *Information and Software Technology* 47, 10 (2005), 643–656. doi:10.1016/j.infsof.2004.11.005
- [22] Barbara Kitchenham et al. 2007. Guidelines for performing systematic literature reviews in software engineering. (2007).
- [23] Jens Knodel and Matthias Naab. 2016. *How to Perform the Architecture Compliance Check (ACC)?* Springer International Publishing, Cham, 83–94. doi:10.1007/978-3-319-34177-4_8
- [24] Jens Knodel and Daniel Popescu. 2007. A Comparison of Static Architecture Compliance Checking Approaches. In *2007 Working IEEE/IFIP Conference on Software Architecture (WICSA'07)*. 12–12. doi:10.1109/WICSA.2007.1
- [25] Christoph Krieger, Uwe Breitenbücher, Michael Falkenthal, Frank Leymann, Vladimir Yussupov, and Uwe Zdun. 2020. Monitoring Behavioral Compliance with Architectural Patterns Based on Complex Event Processing. In *Service-Oriented and Cloud Computing*, Antonio Brogi, Wolf Zimmermann, and Kyriakos Kritikos (Eds.). Springer International Publishing, Cham, 125–140.
- [26] Luka Lelovic, Michael Mathews, Amr Elsayed, Tomas Cerny, Karel Frajtak, Pavel Tisnovsky, and Davide Taibi. 2022. Architectural languages in the microservice era: a systematic mapping study. In *Proceedings of the Conference on Research in Adaptive and Convergent Systems (Virtual Event, Japan) (RACS '22)*. Association for Computing Machinery, New York, NY, USA, 39–46. doi:10.1145/3538641.3561486
- [27] James Lewis and Martin Fowler. 2014. Microservices: A Definition of a New Architectural Term. <https://martinfowler.com/articles/microservices.html>. Accessed: 2025-11-11.
- [28] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. 2022. Understanding software architecture erosion: A systematic mapping study. *Journal of Software: Evolution and Process* 34, 3 (2022), e2423. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.2423 doi:10.1002/smr.2423
- [29] Gail C Murphy, David Notkin, and Kevin Sullivan. 1995. Software reflexion models: Bridging the gap between source and high-level models. In *Proceedings of the 3rd ACM SIGSOFT symposium on Foundations of software engineering*. 18–28.
- [30] Sam Newman. 2021. *Building microservices: designing fine-grained systems*. "O'Reilly Media, Inc.".
- [31] Evangelos Ntontos, Uwe Zdun, Ghareeb Falazi, Uwe Breitenbücher, and Frank Leymann. 2022. Assessing Architecture Conformance to Security-Related Practices in Infrastructure as Code Based Deployments. In *2022 IEEE International Conference on Services Computing (SCC)*. 123–133. doi:10.1109/SCC55611.2022.00029
- [32] Evangelos Ntontos, Uwe Zdun, Ghareeb Falazi, Uwe Breitenbücher, and Frank Leymann. 2023. Detecting and Resolving Coupling-Related Infrastructure as Code Based Architecture Smells in Microservice Deployments. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*. 201–211. doi:10.1109/CLOUD60044.2023.00031
- [33] Evangelos Ntontos, Uwe Zdun, Konstantinos Plakidas, and Sebastian Geiger. 2021. Evaluating and Improving Microservice Architecture Conformance to Architectural Design Decisions. In *Service-Oriented Computing*, Hakim Hacid, Odej Kao, Massimo Mecella, Naouel Moha, and Hye-young Paik (Eds.). Springer International Publishing, Cham, 188–203.
- [34] Evangelos Ntontos, Uwe Zdun, Konstantinos Plakidas, and Sebastian Geiger. 2021. Semi-automatic Feedback for Improving Architecture Conformance to Microservice Patterns and Practices. In *2021 IEEE 18th International Conference on Software Architecture (ICSA)*. 36–46. doi:10.1109/ICSA51549.2021.00012
- [35] Evangelos Ntontos, Uwe Zdun, Konstantinos Plakidas, Sebastian Meixner, and Sebastian Geiger. 2020. Assessing Architecture Conformance to Coupling-Related Patterns and Practices in Microservices. In *Software Architecture*, Anton Jansen, Ivano Malavolta, Henry Muccini, Ipek Ozkaya, and Olaf Zimmermann (Eds.). Springer International Publishing, Cham, 3–20.
- [36] Evangelos Ntontos, Uwe Zdun, Konstantinos Plakidas, Sebastian Meixner, and Sebastian Geiger. 2020. Metrics for Assessing Architecture Conformance to

- Microservice Architecture Patterns and Practices. In *Service-Oriented Computing*, Eleanna Kafeza, Boualem Benatallah, Fabio Martinelli, Hakim Hacid, Athman Bouguettaya, and Hamid Motaahri (Eds.). Springer International Publishing, Cham, 580–596.
- [37] Evangelos Ntontos, Uwe Zdun, Jacopo Soldani, and Antonio Brogi. 2022. Assessing Architecture Conformance to Coupling-Related Infrastructure-as-Code Best Practices: Metrics and Case Studies. In *Software Architecture*, Ilias Gerostathopoulos, Grace Lewis, Thais Batista, and Tomáš Bureš (Eds.). Springer International Publishing, Cham, 101–116.
- [38] Leonardo Passos, Ricardo Terra, Marco Tulio Valente, Renato Diniz, and Nabor Mendonça. 2009. Static architecture-conformance checking: An illustrative overview. *IEEE software* 27, 5 (2009), 82–89.
- [39] Ednilson Geraldo Rossi, Valter Vieira Camargo, and Daniel San Martin. 2026. Replication Package for "Architecture Conformance Checking in Microservice-based Systems: A Systematic Review". doi:10.5281/zenodo.22969480
- [40] Fatima Sabir, Francis Palma, Ghulam Rasool, Yann-Gaël Guéhéneuc, and Naouel Moha. 2019. A systematic literature review on the detection of smells and their evolution in object-oriented and service-oriented systems. *Software: Practice and Experience* 49, 1 (2019), 3–39. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.2639 doi:10.1002/spe.2639
- [41] Johnny Saldaña. 2013. *The Coding Manual for Qualitative Researchers* (2 ed.). SAGE Publications, London, UK.
- [42] Larissa Schmid, Tobias Hey, Martin Armbruster, Sophie Corallo, Dominik Fuchß, Jan Keim, Haoyu Liu, and Anne Kozirolek. 2025. Software Architecture Meets LLMs: A Systematic Literature Review. *arXiv preprint arXiv:2505.16697* (2025).
- [43] Simon Schneider, Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, Riccardo Scandariato, and Davide Taibi. 2025. Comparison of static analysis architecture recovery tools for microservice applications. *Empirical Software Engineering* 30, 5 (2025), 128.
- [44] Apitchaka Singjai and Uwe Zdun. 2022. API Description-Based Conformance Assessment of Architectural Design Decision. In *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 59–68. doi:10.1109/SOSE55356.2022.00013
- [45] Apitchaka Singjai and Uwe Zdun. 2022. Conformance assessment of Architectural Design Decisions on API endpoint designs derived from domain models. *Journal of Systems and Software* 193 (2022), 111433. doi:10.1016/j.jss.2022.111433
- [46] Apitchaka Singjai and Uwe Zdun. 2022. Conformance Assessment of Architectural Design Decisions on the Mapping of Domain Model Elements to APIs and API Endpoints. In *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*. 76–83. doi:10.1109/ICSA-C54293.2022.00058
- [47] Jacopo Soldani, Giuseppe Muntoni, Davide Neri, and Antonio Brogi. 2021. The μ TOSCA toolchain: Mining, analyzing, and refactoring microservice-based architectures. *Software: Practice and Experience* 51, 7 (2021), 1591–1621. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.2974 doi:10.1002/spe.2974
- [48] Chang-Ai Sun, Yufei Gong, Meng Li, Luo Xu, Jun Han, and Yanbo Han. 2024. Detecting Inconsistencies in Microservice-Based Systems: An Annotation-Assisted Scenario-Oriented Approach. *IEEE Transactions on Services Computing* 17, 5 (2024), 2194–2209. doi:10.1109/TSC.2024.3399652
- [49] Chang-Ai Sun, Yufei Gong, Meng Li, Luo Xu, Jun Han, and Yanbo Han. 2024. Detecting Inconsistencies in Microservice-Based Systems: An Annotation-Assisted Scenario-Oriented Approach. *IEEE Transactions on Services Computing* 17, 5 (2024), 2194–2209. doi:10.1109/TSC.2024.3399652
- [50] Ricardo Terra and Marco Tulio Valente. 2009. A dependency constraint language to manage object-oriented software architectures. *Softw. Pract. Exper.* 39, 12 (Aug. 2009), 1073–1094.
- [51] Rainer Weinreich and Georg Buchgeher. 2014. Automatic Reference Architecture Conformance Checking for SOA-Based Software Systems. In *2014 IEEE/IFIP Conference on Software Architecture*. 95–104. doi:10.1109/WICSA.2014.22
- [52] Claes Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*. 1–10.
- [53] Uwe Zdun, Elena Navarro, and Frank Leymann. 2017. Ensuring and Assessing Architecture Conformance to Microservice Decomposition Patterns. In *Service-Oriented Computing*, Michael Maximilien, Antonio Vallecillo, Jianmin Wang, and Marc Oriol (Eds.). Springer International Publishing, Cham, 411–429.
- [54] Uwe Zdun, Pierre-Jean Queval, Georg Simhandl, Riccardo Scandariato, Somik Chakravarty, Marjan Jelić, and Aleksandar Jovanović. 2024. Detection Strategies for Microservice Security Tactics. *IEEE Transactions on Dependable and Secure Computing* 21, 3 (2024), 1257–1273. doi:10.1109/TDSC.2023.3276487
- [55] Chenxing Zhong, He Zhang, Huang Huang, Zhikun Chen, Chao Li, Xiaodong Liu, and Shanshan Li. 2024. DOMICO: Checking conformance between domain models and implementations. *Software: Practice and Experience* 54, 4 (2024), 595–616. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.3272 doi:10.1002/spe.3272